



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO
GENETIC ALGORITHMS



PRÁCTICA 5
GA'S FOR COMBINATORIAL OPTIMIZATION

NOMBRE DEL ALUMNO: GARCÍA QUIROZ GUSTAVO IVAN

GRUPO: 6CV2

NOMBRE DEL PROFESOR: ROSAS TRIGUEROS JORGE LUIS

FECHA DE REALIZACIÓN DE LA PRÁCTICA: 08/09/2024

FECHA DE ENTREGA DEL REPORTE: 22/09/2024

Índice

Marco teórico.....	1
Problema del viajante.....	1
Caso práctico.....	1
Material y equipo.	3
Hardware	3
Software.....	3
Desarrollo de la práctica.....	4
Actividades desarrolladas	4
Configuración del Problema.....	4
Implementación del Algoritmo Genético.....	5
Componentes del Algoritmo Genético	6
Población inicial	6
Evaluación de Fitness.....	6
Operadores Genéticos.....	6
Elitismo	6
Desafíos y soluciones	6
Resultados obtenidos.....	8
Conclusiones y recomendaciones.....	9
Bibliografía.	10

Marco teórico.

Problema del viajante

Dado un conjunto de ciudades y la distancia entre cada par de ciudades, el problema es encontrar la ruta más corta posible que visite cada ciudad exactamente una vez y regrese al punto de partida. Nótese la diferencia entre el ciclo hamiltoniano y el TSP. El problema del ciclo hamiltoniano es encontrar si existe un recorrido que visite cada ciudad exactamente una vez. Aquí sabemos que existe un recorrido hamiltoniano (porque el gráfico está completo) y, de hecho, existen muchos de esos recorridos, el problema es encontrar un ciclo hamiltoniano de peso mínimo.

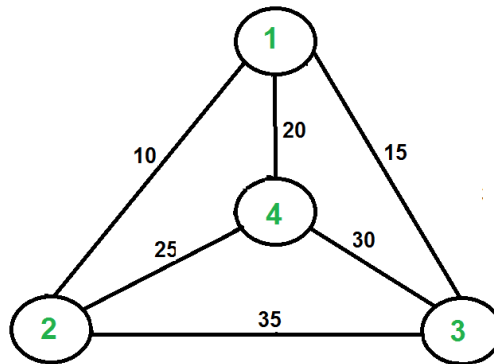


Figura 1 Grafo de la ciudades.

Por ejemplo, considere el gráfico que se muestra en la figura del lado derecho. Un viaje TSP en el gráfico es 1-2-4-3-1. El costo del viaje es $10+25+30+15$, que es 80. El problema es un famoso problema NP-hard. No existe una solución conocida en tiempo polinómico para este problema. Las siguientes son diferentes soluciones para el problema del viajante de comercio.

Caso práctico

En el problema se presentan $N!$ rutas posibles, aunque se puede simplificar ya que dada una ruta nos da igual el punto de partida y esto reduce el número de rutas a examinar en un factor N quedando $(N-1)!$. Como no importa la dirección en que se desplace el viajante, el número de rutas a examinar se reduce nuevamente en un factor 2. Por lo tanto, hay que considerar $(N-1)!/2$ rutas posibles.

En la práctica, para un problema del viajante con 5 ciudades hay $(5-1)!/2=12$ rutas diferentes y no necesitamos un ordenador para encontrar la mejor ruta, pero apenas aumentamos el número de ciudades las posibilidades crece factorialmente:

- Para 10 ciudades hay $(10-1)!/2=181.440$ rutas diferentes
- Para 30 ciudades hay más de $4 \cdot 10^{30}$ rutas posibles. Un ordenador que calcule un millón de rutas por segundo necesitaría 10^{17} años para resolverlo. Dicho de otra forma, si se hubiera comenzado a calcular al

comienzo de la creación del universo (hace unos 13.400 millones de años) todavía no se habría terminado.

Puede comprobarse que por cada ciudad nueva que incorporemos, el número de rutas se multiplica por el factor N y crece factorialmente. Por ello el problema pertenece a la clase de problemas NP-completo

Material y equipo.

Para esta práctica se usaron las siguientes herramientas de software y hardware necesarias para realizar la práctica.

Hardware

- Computadora.

Software

- Google colabory.
- Librería NumPy.
- Phyton 3.

Desarrollo de la práctica.

La práctica se enfoca en la implementación de un Algoritmo Genético (GA) para resolver el Problema del Viajante (TSP) con 10 ciudades. El desarrollo utiliza Python como lenguaje de programación junto con la librería NumPy para el manejo eficiente de matrices y operaciones numéricas.

Actividades desarrolladas

Configuración del Problema

El problema se configura mediante una matriz de distancias M de dimensiones 11×11 , inicializada con valores base de 2. Se establecen conexiones específicas con distancia 1 entre pares de ciudades determinados. La implementación se estructura en una clase denominada `GeneticAlgorithm` que acepta parámetros configurables incluyendo el tamaño de la población, número de generaciones, tasa de mutación y tamaño de élite.

Se definió una matriz de distancias M (11×11) utilizando NumPy, donde la mayoría de las distancias entre ciudades son 2, excepto por algunas conexiones específicas con distancia 1.

```
array([[2., 2., 2., 2., 2., 2., 2., 2., 2., 2., 2.],
       [2., 2., 2., 1., 2., 2., 2., 2., 2., 2., 2.],
       [2., 2., 2., 2., 1., 2., 2., 2., 2., 2., 2.],
       [2., 2., 2., 2., 2., 1., 2., 2., 2., 2., 2.],
       [2., 2., 2., 2., 2., 2., 1., 2., 2., 2., 2.],
       [2., 2., 2., 2., 2., 2., 2., 1., 2., 2., 2.],
       [2., 2., 2., 2., 2., 2., 2., 2., 1., 2., 2.],
       [2., 2., 2., 2., 2., 2., 2., 2., 2., 1., 2.],
       [2., 2., 2., 2., 2., 2., 2., 2., 2., 2., 1.],
       [2., 2., 1., 2., 2., 2., 2., 2., 2., 2., 2.],
       [2., 2., 2., 2., 2., 2., 2., 2., 2., 2., 2.]])
```

Figura 2 Matriz de distancias M (11×11).

Se implementó una clase `GeneticAlgorithm` con los siguientes parámetros configurables:

- Tamaño de población: 100 individuos
- Número de generaciones: 200
- Tasa de mutación: 0.1
- Tamaño de élite: 5

```
ga = GeneticAlgorithm(population_size=100,
                      generations=200,
                      mutation_rate=0.1,
                      elite_size=5)
```

Figura 3 Parámetros configurables del Algoritmo Genético.

Implementación del Algoritmo Genético

La implementación del algoritmo se basa en una clase que encapsula todos los componentes necesarios. Los parámetros se establecen con valores predeterminados de 100 individuos para el tamaño de población, 200 generaciones para el ciclo evolutivo, una tasa de mutación de 0.1 y un tamaño de élite de 5 individuos. La representación de cada solución candidata consiste en una permutación de números enteros del 1 al 10, donde cada número corresponde a una ciudad específica en la ruta.

```
solve(self) -> Tuple[List[int], float]:
population = self.create_initial_population()
best_route = None
best_fitness = float('-inf')

for generation in range(self.generations):
    # Calcular fitness para toda la población
    fitnesses = [self.calculate_fitness(route) for route in population]
    # Guardar el mejor de esta generación
    current_best = max(range(len(population)), key=lambda x: fitnesses[x])
    if fitnesses[current_best] > best_fitness:
        best_fitness = fitnesses[current_best]
        best_route = population[current_best]
    # Crear nueva población
    new_population = []

    # Elitismo
    sorted_population = [x for _, x in sorted(zip(fitnesses, population), reverse=True)]
    new_population.extend(sorted_population[:self.elite_size])

    # Generar el resto de la población
    while len(new_population) < self.population_size:
        parent1, parent2 = self.select_parents(population, fitnesses)
        child = self.crossover(parent1, parent2)
        child = self.mutate(child)
        new_population.append(child)

    population = new_population
    if generation % 10 == 0:
        print(f"Generación {generation}: Mejor fitness = {best_fitness} Mejor ruta ={best_route}")

return best_route, 1/best_fitness
```

Figura 4 Implementación del Algoritmo Genético.

Componentes del Algoritmo Genético

Población inicial

La población inicial se genera mediante el método `create_initial_population()`, que produce permutaciones aleatorias válidas de las ciudades. El cálculo del fitness se realiza mediante una función que suma las distancias entre ciudades consecutivas en la ruta y retorna el inverso de esta suma, estableciendo así una relación donde menor distancia corresponde a mayor fitness.

Evaluación de Fitness

Se desarrollo el método `calculate_fitness()` que: Calcula la distancia total de una ruta sumando las distancias entre ciudades consecutivas. Retorna el inverso de la distancia total ($1/\text{distancia}$) para maximizar el fitness.

$$\text{Fitness} = \frac{1}{\text{distancia}}$$

Selección y Reproducción

El proceso de selección implementa un método de torneo que selecciona tres individuos al azar y escoge el mejor basado en su valor de fitness. Este proceso se repite para obtener dos padres que participarán en la reproducción. El operador de cruce implementado corresponde al Ordered Crossover (OX), que preserva subsecuencias de ciudades de un padre mientras mantiene el orden relativo de las ciudades del otro padre en las posiciones restantes.

Operadores Genéticos

La mutación se implementa mediante un operador de intercambio que, con una probabilidad de 0.1, selecciona dos posiciones aleatorias en la ruta y intercambia las ciudades correspondientes. Este mecanismo permite mantener la diversidad en la población y explorar nuevas regiones del espacio de búsqueda.

- Cruce (Crossover): Se implementó el Ordered Crossover (OX) en el método `crossover()`. Preserva el orden y posición de algunos elementos de un padre. Completa el resto con elementos del segundo padre manteniendo su orden relativo.
- Mutación: Se implementó la mutación por intercambio en el método `mutate()`. Con una probabilidad de 0.1, intercambia dos ciudades aleatorias en la ruta.

Elitismo

El algoritmo implementa elitismo mediante la preservación de los 5 mejores individuos de cada generación. Este mecanismo asegura que las mejores soluciones encontradas no se pierdan durante el proceso evolutivo y contribuye a la convergencia del algoritmo hacia soluciones óptimas.

Desafíos y soluciones

Entre los desafíos que tuvimos que resolver son la validación de Rutas, la convergencia y el cálculo de Fitness. El primero consistió en garantizar la validez de las rutas generadas, lo cual se resolvió mediante el uso apropiado de la función `random.sample()`. El segundo desafío fue la convergencia prematura del algoritmo, que se abordó mediante el ajuste de los parámetros del algoritmo y la implementación del elitismo. El tercer desafío se relacionó con el manejo de las diferentes escalas en el cálculo del fitness, que se resolvió mediante la implementación del inverso de la distancia total como medida de aptitud.

Resultados obtenidos

[illegible]

Figura 5 Resultados para el Problema del Viajante (TSP) con 10 ciudades.

Conclusiones y recomendaciones.

La implementación del Algoritmo Genético para resolver el Problema del Viajante demuestra la efectividad de los métodos evolutivos en la optimización combinatoria. Los resultados obtenidos confirman que la estructura implementada, basada en operadores genéticos específicos y mecanismos de preservación elitista, logra generar soluciones válidas que optimizan las rutas entre las ciudades establecidas.

La combinación de selección, cruce ordenado y mutación por intercambio resultó efectiva para mantener un balance entre la exploración del espacio de búsqueda y la explotación de soluciones prometedoras. El mecanismo de elitismo contribuyó a la preservación de las mejores soluciones encontradas durante el proceso evolutivo.

Para futuras implementaciones, se recomienda ajustar la tasa de mutación y el tamaño de la población podrían ajustarse automáticamente durante la ejecución, respondiendo a métricas como la diversidad de la población o la tasa de mejora de las soluciones.

La implementación de estructuras de datos más eficientes para el almacenamiento y manipulación de las rutas podría mejorar el rendimiento computacional del algoritmo. El uso de arrays NumPy especializados o estructuras de datos optimizadas para permutaciones podría reducir el tiempo de ejecución en instancias más grandes del problema.

El desarrollo de una interfaz gráfica para la visualización de las rutas y la evolución del algoritmo facilitaría el análisis y la comprensión del proceso de optimización. La representación visual de las soluciones y las métricas de rendimiento proporcionaría información para el ajuste y mejora del algoritmo.

Bibliografía.

- Improve, G. (2013, noviembre 3). *Travelling Salesman Problem using Dynamic Programming*. GeeksforGeeks. <https://www.geeksforgeeks.org/travelling-salesman-problem-using-dynamic-programming/>
- Wikipedia contributors. (s/f). *Problema del viajante*. Wikipedia, The Free Encyclopedia. https://es.wikipedia.org/w/index.php?title=Problema_del_viajante&oldid=162673478
- Google colab. (s/f). Google.com. Recuperado el 20 de octubre de 2024, de <https://colab.research.google.com/drive/1qB4J-xTNOd872MYJO6IVyRgh8lcsDSH7>